

Assignment 4 — Distributed Stream Processing
AITU Research Event Analytics with Kafka and Spark

Student full name: Ayazhan Sailauova

Student ID: 240791

Group: BDA-2402

Instructor: Azamat Serek

Submission date: 31.10.2026

Task Code: AITU Research Event Analytics with Kafka and Spark

Git repository: https://github.com/avayashka/RTDA_Assignment4.git

Python version: Python 3.13

1. Work Record

Item	Student entry
Work start date	30.10.2026
Work completion date	31.10.2026
Main libraries / technologies	Python, pandas, matplotlib, psutil, kafka-python, PySpark (Structured Streaming), Apache Kafka (Docker), Jupyter
Source files / notebook	Assignment4.ipynb; src/common.py, kafka_producer.py, kafka_consumer.py, spark_streaming_job.py, load_test.py; docker-compose.yml
Test / experiment files	data/events_initial.csv, data/events_extended.csv, data/stream_input/*.csv; results/*.csv, results/*.png, results/summary.json
Final repository commit/version	https://github.com/avayashka/RTDA_Assignment4.git

2.1 Streaming Architecture

Required evidence	Student response / reference
Technical explanation	Three producers (one per research group) publish JSON events with key = research_group to topic research-events (3 partitions). Consumer group research-analytics runs up to 3 Python consumers, one per partition; each validates events, sends invalid ones to research-events-dlq and keeps partial aggregates. The aggregation layer merges the partial sums; Spark Structured Streaming reads the same topic as an independent consumer for window x job_type analytics. Monitoring watches lag, throughput/latency, DLQ size and window alerts. Failures: replication + committed offsets + rebalancing + Spark checkpoints. Scaling: more partitions and consumers.
Code / file / function / configuration reference	docs/architecture.png; docker-compose.yml (apache/kafka:3.8.0, KRaft, localhost:9092, Kafka UI :8080); src/common.py: TOPIC='research-events', NUM_PARTITIONS=3, DLQ_TOPIC='research-events-dlq'; src/kafka_producer.py: ensure_topics()
Result / calculation / experiment evidence	Configuration: topic research-events, 3 partitions, replication factor 1 (local), DLQ topic, consumer group research-analytics (docker-compose.yml, src/common.py). As allowed by the instructor, the producer/consumer flow was executed with an in-memory queue in place of the broker: 12 messages -> 10 valid, 2 sent to the invalid path; Spark processed the same stream in 9 micro-batches. Figure 1 shows the design.
Student-specific decision or observation	Key = research_group with 3 partitions (=

	number of groups): keeps per-group order and spreads load. Invalid events go to a DLQ instead of being dropped, so they can be inspected and replayed.

2.2 Kafka Producer and Consumer

Required evidence	Student response / reference
Technical explanation	The statistics are computed in plain Python without Kafka: a producer thread serialises each event to JSON and puts it on a queue.Queue (stand-in for the topic); a consumer thread deserialises, validates the schema (all 7 fields present, no extra fields, timestamp HH:MM:SS, job_type and status from the allowed sets, duration > 0, 0 <= cpu_pct <= 100) and aggregates. The same logic is available with a real broker in kafka_producer.py / kafka_consumer.py.
Code / file / function / configuration reference	Assignment4.ipynb Part B: producer(), consumer(), statistics cell, pandas cross-check; src/common.py validate_event(); src/kafka_consumer.py update(), merge(); results/partB_job_type.csv, partB_cpu_by_group.csv, partB_charts.png
Result / calculation / experiment evidence	12 messages -> 10 valid, 2 rejected (E998, E999). Completed 8, failed 2. ml 2/1, avg 52.33 s; image 3/0, 77.67 s; text 2/0, 20.00 s; simulation 1/1, 107.50 s. Avg CPU: G01 68.50 %, G02 62.00 %, G03 86.33 %. Independent pandas cross-check passed.
Student-specific decision or observation	Both failed jobs (E004, E007) belong to G03 and ran at CPU >= 89 %; G03 also has the highest average CPU - failures correlate with CPU saturation. The consumer stores sums and counts, not averages, so results from several consumers can be merged exactly.

2.3 Windowed Stream Processing

Required evidence	Student response / reference
Technical explanation	Tumbling 5-s windows on event time: window start = floor(t/5)*5, interval [start, start+5). Events are processed one by one in arrival order. Watermark = max event time seen - 5 s; when the watermark passes a window's end the window is emitted, and an event for an already closed window would be dropped as late. Per window:

	count, failure rate = failed/count, average CPU. The stream is extended with synthetic events generated from the profile of the supplied data (job mix, mean duration per type, CPU-duration regression, CPU-dependent failure probability, ~5 % out-of-order arrivals). This is simulation of additional traffic, not prediction.
Code / file / function / configuration reference	Assignment4.ipynb Part C: class TumblingWindowProcessor (process, _emit, flush); src/common.py generate_extended_stream(seed=240791); results/partC_windows_extended.csv, partC_windows_10_events.csv, partC_windows.png
Result / calculation / experiment evidence	108 events -> 24 windows. 14:00:00-05: 4 events, 25 %, 66.25; 14:00:05-10: 5 events, 20 %, 75.20; 14:00:10-15: 2 events (E010, E011), 0 %, 74.50. Out-of-order event E045 placed in the correct window, 0 late events dropped. Streaming result equals batch groupby for all 24 windows. Alerts: 14:00:25-30 (100 % failures), 14:00:35-40 (CPU 95.33), 14:01:25-30 (CPU 87.50), 14:01:50-55 (100 % failures).
Student-specific decision or observation	Windows with only 1-2 events produce extreme failure rates (100 %), so an alert rule should also require a minimum count (e.g. >= 3 events). Seed = student ID makes my extended stream individual.

2.4 Spark Structured Streaming

Required evidence	Student response / reference
Technical explanation	Spark reads the extended stream as a streaming source: 9 CSV files of 12 events in data/stream_input, maxFilesPerTrigger = 1, so each file is one micro-batch. Transformation: to_timestamp (date added) -> filter invalid rows -> withWatermark('event_time', '10 seconds') -> groupBy(window(event_time, '5 seconds'), job_type) -> count('*') as count, round(avg(duration_sec), 2) as avg_duration. Sink: memory table job_type_windows, output mode complete, queried with SQL. The standalone job also supports a Kafka source (from_json on the value column).
Code / file / function / configuration reference	Assignment4.ipynb Part D; src/spark_streaming_job.py: read_stream(), windowed_job_type_stats(), prepare_stream_files(); config: master local[3],

	spark.sql.shuffle.partitions=3, spark.sql.session.timeZone=UTC, package spark-sql-kafka-0-10_2.12:3.5.3 for the Kafka source.
Result / calculation / experiment evidence	Spark 3.5.3 output: 65 (window, job_type) rows, e.g. 14:00:00-05: image 1/75.0, ml 1/42.0, simulation 1/95.0, text 1/18.0; 14:00:05-10: ml 2/57.5; 14:00:10-15: image 2/68.5. Notebook check: 'Spark output identical to pandas expectation: True (65 rows)'. Query progress: 9 micro-batches (last batchId 8), 12 input rows per batch, triggerExecution 1090 ms, state store 65 rows in 3 instances, watermark 14:01:34, numRowsDroppedByWatermark 0.
Student-specific decision or observation	shuffle.partitions reduced from the default 200 to 3 - with 200 every tiny micro-batch schedules 200 tasks. Complete mode is used because the memory sink must hold the full result; in production, update mode with the watermark keeps state small.

2.5 Throughput and Latency

Required evidence	Student response / reference
Technical explanation	Synthetic load: a producer thread sends events at a fixed rate for 10 s (catch-up pacing); a consumer thread performs the real pipeline work (JSON decode, schema validation, window update). Latency = consumer finish time - producer creation time; throughput = processed / elapsed; CPU = process CPU (psutil, 100 % = one core). Levels 10, 100, 500 plus 2000 and 5000 ev/s, and a capacity test.
Code / file / function / configuration reference	src/load_test.py: run_inproc_level(), run_inproc_capacity(), run_kafka_level(), bottleneck_label(); Assignment4.ipynb Part E; results/partE_load_levels.csv, partE_capacity.csv, partE_performance.png
Result / calculation / experiment evidence	10 ev/s: 10.1 ev/s, 0.488 ms, 1.6 % CPU; 100 ev/s: 100.1, 0.349 ms, 2.2 %; 500 ev/s: 500.1, 0.302 ms, 2.5 %. 2000 and 5000 ev/s also fully processed (CPU 4.4 % and 9.4 %). Capacity 103,266 ev/s (headroom at 500 ev/s: 207x).
Student-specific decision or observation	Average latency is highest at 10 ev/s (0.488 ms) and lowest at 500 ev/s (0.302 ms): at low rates the consumer thread sleeps between events and must be woken for each one. Spark adds far more latency per result (about 1.1 s per micro-batch in my run) than the Python processor (< 1 ms per

	event).

2.6 Engineering Interpretation

Required evidence	Student response / reference
Technical explanation	Main bottleneck: the single Python consumer process - all processing runs on one core because of the GIL. While input rate < capacity, throughput equals input rate and latency stays sub-millisecond; above capacity the queue (or Kafka consumer lag) grows without bound and latency rises continuously. With Kafka the next limits are per-partition consumer speed and the network/broker round trip. The system therefore scales by adding partitions and consumer processes, not threads.
Code / file / function / configuration reference	src/load_test.py (batch=True/False); results/partE_capacity.csv; Assignment4.ipynb Part E capacity cell
Result / calculation / experiment evidence	Per-event consumer: 103,266 ev/s at 97.6 % CPU (one core of 12); batch-draining consumer: 102,823 ev/s (-0.4%). At 5000 ev/s CPU is only 9.4 %. Spark: ~1.09 s per micro-batch of 12 rows (fixed overhead: WAL commit 220 ms, state commit 681 ms).
Student-specific decision or observation	My measurement rejected batch draining (-0.4 %, within noise), so the justified optimisation is horizontal scaling: run one consumer process per partition (3 partitions -> 3 cores), since one core is saturated while 11 are idle. For Spark, larger micro-batches (or a trigger interval) amortise the fixed ~1 s commit overhead.

3. Individual Work and Anti-Plagiarism Record

Individual evidence	Student entry
My unique technical/design decision	Partition key = research_group with 3 partitions, a dead-letter topic for invalid events, and consumers that keep partial sums so the aggregation layer can merge them exactly.
My own test/experiment	Extended stream (seed 240791) with out-of-order event E045 to test event-time windows and watermark; capacity test per-event vs batch-draining consumer (103 266 vs 102 823 ev/s - batching rejected).
Problem/bug I personally encountered and solved	Spark would not start on Windows: (1) Python 3.14 is not supported by PySpark 3.5.3 -> created a Python

	3.11 venv; (2) JAVA_GATEWAY_EXITED / Py4JJavaError because Java 25 was installed -> installed Temurin 17 and set JAVA_HOME in the notebook; (3) readStream failed without Hadoop native files -> installed winutils.exe and hadoop.dll in C:\hadoop\bin and set HADOOP_HOME.
Exact code/repository/commit reference	https://github.com/avayashka/RTDA_Assignment4.git
One limitation of my implementation	Single local broker with replication factor 1 (no real fault tolerance); the extended data is synthetic, derived from only 10 events; the in-process load test does not include network cost.
One improvement I propose	Run three consumer processes (one per partition) and add a Prometheus/Grafana dashboard with consumer-lag alerts; require >= 3 events in a window before raising a failure-rate alert.

4. Student Declaration of Originality

I confirm that this submission represents my own work. I have not copied another student's source code, calculations, tables, architecture, plots, screenshots, logs, results, or written explanations. Any external material or reused code has been identified. I understand that the instructor may request a live demonstration, code modification, reproduction of calculations, or explanation of any submitted component.